
Analysis on How AI is Being Applied in Flaky Test Detection

Prepared By:

Name: Bandharam, Tharun Goud

ID: 2457134

```
def test_model_accuracy(input_data):  
    if response.status_code == 200:  
        coder.solve_ext()  
  
    if response.status_code == 280:  
        result = vendor1()  
        return result, sensor_roster(input_data);  
    }  
    for layer in model.layers:  
        expect(result).toBeTrue()  
        return next  
    }  
}
```

C++

```
def test_model():  
    test_suite = C++;  
    var_mount = model;  
    variable = JSON  
}
```

```
def test_model_accuracy(input_data):  
    init.s = 0;  
    if response.status_code == 200:  
        for layer in model.layers:  
            var_suit = test_suit == 0;  
            response.status_code(input_data);  
            new result;  
            return 1;  
        }  
    test_suite = NeuralNet(input_data);(  
        expect(result).toBeTrue()  
        return result;  
    )  
}
```



Introduction



Automated testing plays a critical role in maintaining high-quality software standards.



Flaky tests: Tests that produce inconsistent results without code changes, undermining trust.



These inconsistencies increase debugging overhead and diminish overall confidence in automation suites.



AI is now vital for intelligently identifying, clustering, and analyzing these flaky behaviors.



What are Flaky Tests?



Tests that pass and fail intermittently.



Failures occur without changes to application code.



Common Causes:

- Timing & Network Instability
- Environment Issues
- Test Dependencies

VISUAL EXAMPLE: INTERMITTENCY

Run 1



PASS

Run 2



FAIL

Run 3



PASS

Identical code, inconsistent outcomes

Challenges Caused by Flaky Tests



Increased Debugging Effort

Inconsistent failures require deep manual investigation.



Higher Maintenance

Constant updates to test scripts without code changes.



Reduced Confidence

Teams ignore test results, leading to "false negatives."



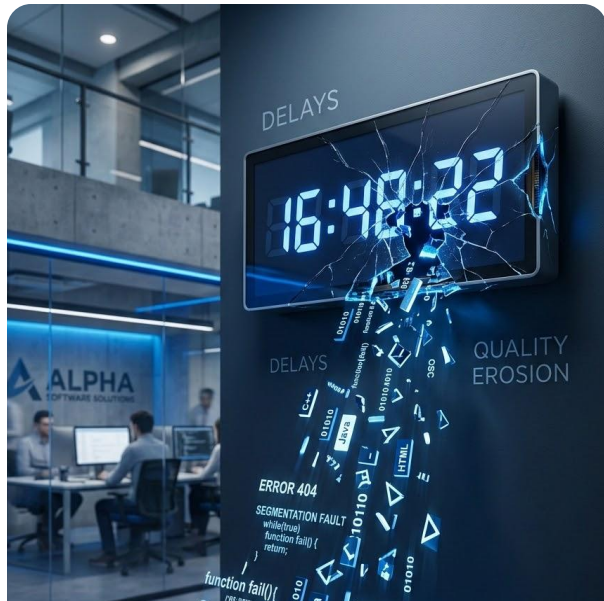
Defect Masking

Harder to distinguish flakes from real application bugs.



CI/CD Pipeline Delays

Repeated executions slow down release cycles.



IMPACT SUMMARY

Erosion of software quality and extended release timelines due to testing friction.

Traditional Approaches to Flaky Test Detection

Common Manual Methods



Repeated Execution

Re-running failed test cases multiple times.



Report Review

Reviewing test execution reports and dashboards.



Log Analysis

Analyzing logs, traces, and screenshots.



Historical Comparison

Comparing results across past test runs.



Environmental Investigation

Investigating infrastructure, network, or timing-related issues manually.

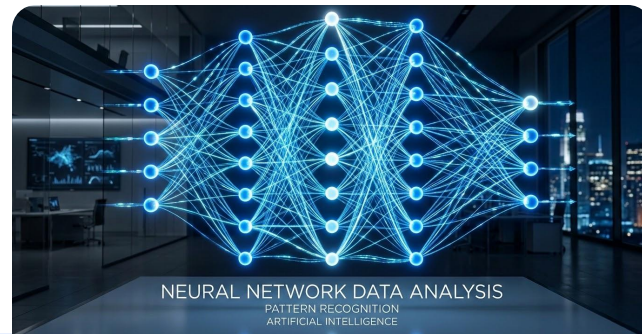
LIMITATIONS

- **Time-consuming:** Manual triage significantly slows down CI/CD cycles.
- **Resource Intensive:** Requires deep technical effort from engineers.
- **Scalability Gap:** Impossible to sustain for large-scale automation suites.

How AI is Applied in Flaky Test Detection

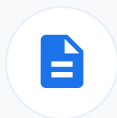
AI identifies flaky tests by analyzing historical execution data to recognize hidden patterns.

- Analyze test execution history.
- Detect inconsistent test behavior.
- Identify recurring failure patterns.



- Classify failures based on similarity.
- Assist in root cause investigation.

Test Results



Historical Data



AI Pattern
Recognition



Flaky
Detection



Insights



Real-World Example

A login test is executed multiple times during automated regression testing. Although no changes have been made to the application code, the test produces inconsistent results.

EXECUTION HISTORY

Run 1 **PASS**

Run 2 **FAIL**

Run 3 **PASS**

Run 4 **PASS**

Run 5 **FAIL**

AI ANALYSIS FLOW

Execution History



Pattern Analysis



Anomaly Detection



Flaky Test Flagged

AI DETECTION PROCESS

By analyzing historical execution results, AI identifies recurring intermittent failures and detects patterns that indicate test instability.

Based on this analysis, the test is flagged as a potential flaky test for further investigation.



Benefits and Conclusion

Key Benefits of AI Integration



Faster Identification

Rapidly flags unstable tests



Reduced Effort

Minimizes manual triage



Improved Reliability

Consistent automation results



CI/CD Stability

Seamless pipeline execution



Increased Release Confidence

Higher quality software delivered with certainty



Conclusion

AI provides an efficient, scalable approach to identifying unstable tests by analyzing historical execution patterns. As suites grow, AI-driven detection becomes vital for release quality.